INTERNATIONAL JOURNAL OF DIGITAL EARTH
2024, VOL. 17, NO. 1, 2308707
https://doi.org/10.1080/17538947.2024.2308707

OPEN ACCESS Check for updates

Parallel algorithm for multi-viewpoint viewshed analysis on the GPU grounded in target cluster segmentation

Tianren Xiao [a,b], Jiqiu Deng [a,b], Chengfeng Wen [a,b] and Qiqi Gu [a,b]

[a]Key Laboratory of Metallogenic Prediction of Nonferrous Metals and Geological Environment Monitoring, Central South University, Ministry of Education, Changsha, People's Republic of China; [b]School of Geosciences and Info-Physics, Central South University, Changsha, People's Republic of China

ABSTRACT

Viewshed analysis is a significant method for GIS spatial analysis. The needed computational resources rise sharply with the increase in the number of viewpoints, making viewshed analysis inefficient in multi-viewpoint scenes. Building on the shadow map algorithm, a parallel algorithm for multi-viewpoint viewshed analysis based on target cluster segmentation for large-scale, multi-viewpoint complex three-dimensional scenes was proposed. Target cluster segmentation was achieved using uniform grid division and K-means spatial clustering, and visibility was determined by comparing the depths. The customized graphics processing unit (GPU) rendering pipeline was adopted to execute the algorithm efficiently and in parallel. The experimental results indicated that the efficiency of the proposed algorithm improves by 6.252%, 8.280%, and 9.047% on average for viewshed analysis with 150, 300, and 600 viewpoints, respectively, compared to the algorithm based solely on shadow map. It is also able to eliminate the majority of shadow acne, thus clearly improving accuracy. The algorithm is compatible with the terrain and features while fully utilizing the parallel computational capability of the GPU and avoiding the interpolation of the digital elevation model (DEM). It significantly improves the efficiency of analysis in large-scale and multi-viewpoint scenes.

ARTICLE HISTORY
Received 12 October 2023
Accepted 17 January 2024

KEYWORDS
shadow map; uniform grid division; K-means; fragment shader; GPU rendering pipeline

1. Introduction

Digital Earth, as a multiresolution three-dimensional (3D) representation of the Earth, allows users to locate, visualize, and interpret large amounts of georeferenced information (Annoni et al. 2023). Spatial analysis is one of its most valuable functions. As an important part of spatial analysis, viewshed analysis is widely used in military, communication, forestry, and archaeology applications, such as battlefield simulations (Henrico, Henrico, and Coetzee 2020; Rua, Gonçalves, and Figueiredo 2013), wireless signal coverage (Welscher et al. 2023), forest fire monitoring (Bao et al. 2015; Sanchez-Fernandez et al. 2022), natural landscape analysis (Sobala, Myga-Piątek, and Szypuła 2020), and archaeological site preservation (Jones 2006; Murphy, Gittings, and Crow 2018).

In 3D space, viewshed analysis refers to the process of analyzing one or more observation points in a scene with a certain horizontal view angle, vertical view angle, and visual scope to obtain the set

CONTACT Jiqiu Deng csugis@csu.edu.cn No.932 South Lushan Road, Changsha Hunan 410083, People's Republic of China

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (http://creativecommons.org/licenses/by-nc/4.0/), which permits unrestricted non-commercial use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

of all visible points in the region. From a macro point of view, objects in the 3D digital scene can be roughly divided into terrain and features. Spatial features use terrain as a carrier to create connections between them. Most of the traditional viewshed analysis algorithms are based on the line-of-sight (LOS) principle (Chao et al. 2011; Feng et al. 2015), and the visibility of terrain is calculated by large-scale interpolation of the digital elevation model (DEM). As a result, the relationship between terrain and features is not considered. Moreover, these algorithms are typically executed in the CPU, leading to a decrease in the efficiency in large-scale 3D scenes. Many scenes also have requirements on the number of viewpoints, such as reverse visibility analyses for landscape planning (Beer, Rybár, and Gabániová 2023; Ioannidis et al. 2022), communication base station siting (Baek and Choi 2018), public security surveillance coverage (Han et al. 2019; Wang et al. 2017), urban architecture and green planning (Cimburova and Blumentrath 2022; Sander and Manson 2007; Yu et al. 2016). Therefore, algorithms that can quickly perform large-scale, multi-viewpoint viewshed analysis are promising for use in applications.

With the rise and development of technologies such as deep learning, AI, and virtual reality (Alzubaidi et al. 2021; Park and Kim 2022; Wu et al. 2023), graphics processing units (GPUs) are no longer exclusively used by a small number of gamers but rather have become production tools for researchers in various fields. Instead of the small number of high-frequency, large cores of CPUs, GPUs have thousands of low-frequency, small cores that are optimized for sequential serial operations. These cores can execute similar tasks at the same time, making GPUs far superior to CPUs in parallel computing power. The shadow Map is one of the most popular real-time shadow generation algorithms and was first proposed by Williams et al. in 1978 (Sen, Cammarano, and Hanrahan 2003). It is widely used in rendering-intensive scenes such as 3D games because of its good versatility of execution on GPUs and its ability to efficiently generate real-time shadows in large-scale virtual environments. Some viewshed analysis algorithms based on the shadow map principle have already been introduced, but still exhibit various shortcomings. For example, these algorithms suffer from a lack of unification of terrain and features, or partial utilization of the parallel computing power of GPUs. This paper proposes a parallel algorithm (hereafter referred to collectively as our algorithm) for multi-viewpoint viewshed analysis based on target cluster segmentation for large-scale, multi-viewpoint complex 3D scenes based on the shadow map algorithm. The algorithm achieves parallel viewshed analysis between clusters of multi-viewpoint scenes by uniform grid division of the target space and K-means spatial clustering of the viewpoints. Compared to the viewshed analysis algorithm based solely on shadow map (hereinafter collectively referred to as the shadow map viewshed algorithm), our algorithm achieves improves both efficiency and accuracy. The main contributions of our work can be summarized as follows: (1) A viewshed analysis algorithm for large-scale, multi-viewpoint scenes that can analyze terrain and features simultaneously is proposed based on the shadow map. (2) Target cluster segmentation is introduced into the viewshed analysis algorithm, enabling full utilization of GPU parallel computing capability and achieving improved efficiency. (3) The adaptive depth offset method is incorporated, improving the algorithm's accuracy compared to the shadow map viewshed algorithm.

The rest of the paper is organized as follows: First, various viewshed analysis algorithms are introduced in Section 2. Section 3 describes the details of the proposed algorithm, including uniform grid division, K-means spatial clustering, adaptive depth bias, and customized fragment shader. Section 4 provides an experimental validation of the efficiency and accuracy of the algorithm and discusses the results. Finally, Section 5 summarizes the advantages and disadvantages of the proposed algorithm and suggests future improvements.

2. Related works

LOS-based algorithms are simple in principle and easy to implement but are generally time-consuming due to their high computational cost. Researchers have proposed various optimization algorithms to reduce the number of computations of LOS-based algorithms. Three classical

algorithms for viewshed analysis, R3, R2, and Xdraw, were proposed based on the LOS algorithm (Wm et al. 1998) and achieved improved efficiency. The key slope method was considered a major improvement to the LOS-based algorithm (De Floriani, Marzano, and Puppo 1994). This algorithm continuously calculates the maximum slope value, replacing elevation comparisons with slope comparisons, substantially improving the efficiency. Wang et al. proposed a reference plane method completely different from the LOS-based algorithm that generates a reference plane for visibility computation based on the spatial topology of the viewpoints between the target points (Wang, Robinson, and White 2000; Yanlan 2001). Afterward, Ying et al. proposed a new double incremental method, Wu et al. proposed a partition-based serial algorithm for generating viewshed on massive DEMs (Wu et al. 2007), and Liu et al. proposed an improved algorithm that utilizes the slopes and elevations between the target pixels and the viewpoints to reduce the amount of computation (Liu et al. 2010). However, these algorithms focus on the viewshed analysis of DEM-based terrain and cannot analyze terrain and features simultaneously.

The ever-increasing magnitude of spatial data poses an enormous computational challenge for large-scale viewshed analysis, and general-purpose computing on GPUs provides a new approach to solving these problems (Owens et al. 2007; Zhao, Padmanabhan, and Wang 2013). A modern GPU-based parallel algorithm for viewshed analysis was proposed (Chao et al. 2011) that demonstrated the efficiency and accuracy of the shadow map algorithm. However, this algorithm only considers the scene with a single viewpoint, and some shadow acne is present in the results due to insufficient precision. Then, a parallel GPU-based algorithm for viewshed analysis in 3D digital earth was proposed (Feng et al. 2015) that generates an occlusion for each geometric entity according to LOS and delineates the visibility through the occlusion. Attempts have also been made to convert the LOS algorithm to run in the GPU environment, and a GPU-based parallel Xdraw algorithm has been proposed (Cauchi-Saunders and Lewis 2015). Chen et al. proposed a parallel multipoint viewshed analysis method for urban 3D building scenes based on previous studies (Chen and Chen 2021). However, the algorithm only supports 3D building models and is also limited by the texture space to analyze scenes with more viewpoints. Existing GPU-based viewshed analysis algorithms are inherently parallel because the GPU cores run their respective shader programs independently on each rendering pipeline.

3. Methodology

The algorithm first eliminates extraneous parts of the target space by uniform grid division and determines the 8-neighborhood of the viewpoint using the unit grid. Next, the viewpoints are segmented by clusters through K-means spatial clustering to enable parallel analysis of the clusters. Finally, the visibility of each point is determined by comparing the depths in the scene with different cameras. The algorithm flowchart is as follows shown in Figure 1.

3.1. Uniform grid division

A too large scope will inevitably decrease the overall efficiency in large-scale, multi-viewpoint complex 3D scenes, in which the study area usually reaches dozens or even hundreds of square kilometers. Numerous unnecessary calculations can be avoided by eliminating spatial regions extraneous to the target viewpoint. Meanwhile, a large number of viewpoints means that field of view overlaps are present, making collision detection and proximity query essential. Only the target viewpoint and viewpoints within its field of view will be tested to increase the efficiency of collision detection between the frustums within each cluster. For optimization operations, large 3D games typically use space division algorithms, including uniform grid division, quadtree, octree, k-d tree, and Bsp tree (Moll et al. 2021; You et al. 2022). This study adopts the uniform grid division method to divide the target area because of its significant advantages in collision detection and neighborhood query.

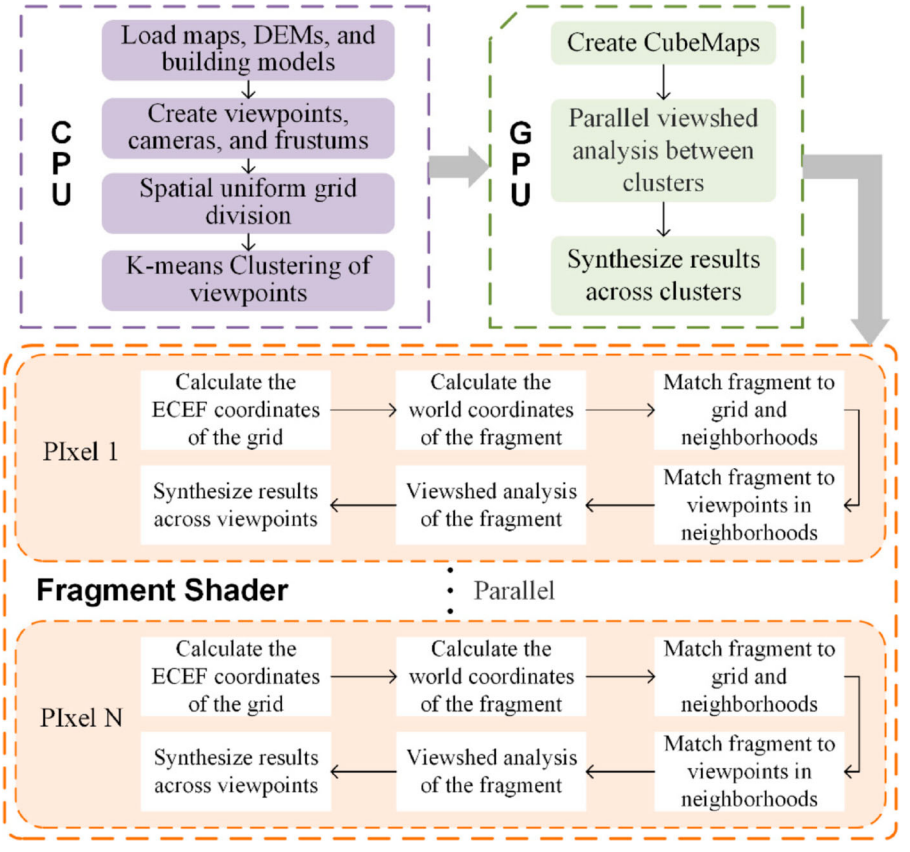


Figure 1. The overall flowchart of our algorithm. The uniform grid division and K-means spatial clustering are executed in the CPU. The viewshed analysis process, which is the main body of the algorithm, is executed in the fragment shader of the GPU.

The camera and the frustum must be created prior to viewshed analysis. First, the point light sources are created at the location of each viewpoint. Then, cameras are created at these locations with light simulating the sight. The perspective projection frustum corresponding to each point light source is defined by six planes. This will produce an effect of large near and small far that is more in line with the imaging law of the human eye (Stamminger and Drettakis 2002). Frustum parameters such as fovx, fovy, aspectRatio, near, and far are set according to the light source and camera information (Figure 2).

Assuming that the set of viewpoints in the study area is $O = \{O_1, O_2, \ldots, O_n\}$, the set of frustums corresponding to its field of view is $F = \{F_1, F_2, \ldots, F_n\}$. To determine the dimensions of the unit grid, it is necessary to first construct the minimum enclosing box for each F_i. Common bounding box algorithms include Axis-aligned bounding box (AABB), Oriented Bounding Box (OBB), and Sphere among which OBB approximates the object more closely than the others and can significantly reduce the size of the enclosing body (Chang, Gorissen, and Melchior 2011). OBB is directional and can always generate the smallest rectangular bounding box along the direction of the principal component of the object. It is more appropriate to use OBB to generate the minimum bounding box for the frustums. The smallest bounding boxes for each of the individual frustums are fairly similar. Therefore, the unit grid can completely wrap around any individual field of view and its size must be slightly larger than the diameter of the largest bounding box. Let the minimum bounding box diameter of any frustum be d_i, and then the

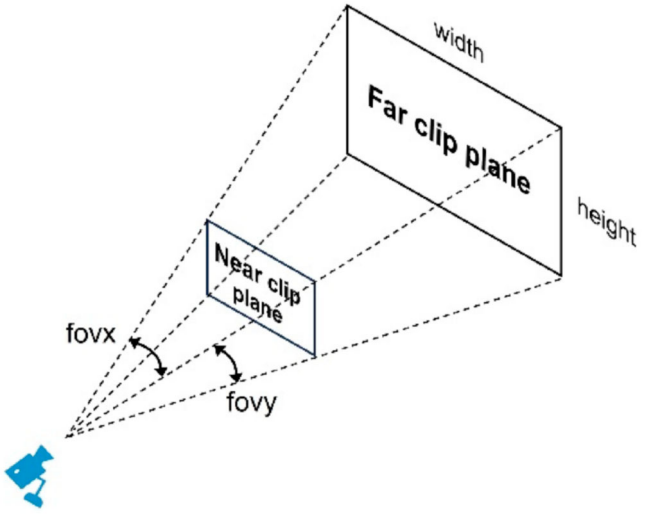


Figure 2. Schematic diagram of the perspective projection frustum. Here, fovx and fovy are the horizontal and vertical angles, aspectRatio is the width-to-height ratio, and near and far are the distances of the two clip planes from the light source.

diameter of the unit grid can be expressed as: $L_{diam} = C_{R1} \times \max\{d_1, d_2, \ldots, d_i\}$, where C_{R1} is the redundancy coefficient.

The next step is to construct the overall bounding box for F to determine the extent of the overall grid corresponding to the study area. Four sides of the unit grids are set to be parallel to the longitude and latitude directions. The respective numbers of the unit grids along the directions are calculated according to the coordinate range of the overall grid and L_{diam}. The study area outside the overall grid is not divided, and this section is also excluded from subsequent analyses. The world coordinate system in Cesium is the ECEF (Earth-Centered, Earth-Fixed). It is necessary to convert the latitudes and longitudes of the grids to geocentric Cartesian coordinates for matching the fragment to the unit grid in the fragment shader. Afterward, each viewpoint is matched to the corresponding grid based on its coordinates, and information regarding the viewpoints and fields of view is stored by grid number.

3.2. Viewpoint spatial clustering

In large-scale, multi-viewpoint 3D scenes, for most viewpoints, there is no intersection between their fields of view. Analysis of these areas as a whole will affect the overall efficiency. Our algorithm is primarily implemented in the fragment shader of the GPU rendering pipeline, and the shader interacts with the CPU for data through the Uniform. If the information on all viewpoints is transmitted to the Uniform simultaneously, it will increase the communication time and the storage capacity of the GPU's constant area. The algorithm first clusters the viewpoints according to their spatial location relationship and then performs the viewshed analysis in parallel by clusters to improve efficiency. Distance-based clustering is highly suitable for the viewpoints in this study. The K-means algorithm is the most famous division-based clustering algorithm and is still the most popular and straightforward clustering algorithm due to its excellent efficiency, good scalability, and ability to maintain good scalability and speed for large datasets (Ikotun et al. 2023). However, this algorithm requires the number of clusters in a given dataset to be specified during the initialization process.

Determining the value of K is an important step in K-means cluster analysis. The number of texture maps transmitted by Uniform in the fragment shader is limited due to WebGL's limitations. Therefore, the number of viewpoints per cluster in the clustering result cannot exceed the maximum

limit MAX, and an inappropriate value of K may lead to poor clustering results. A K that is too large may result in a low average number of viewpoints per cluster, decreasing the efficiency of the overall viewshed analysis. If K is too small, the algorithm is prone to overfitting, resulting in too large a difference in the number of viewpoints between the clusters or even in viewpoints within the clusters exceeding MAX. In our algorithm, K is determined based on the ratio of the total number of viewpoints N to the maximum number of texture images MAX: $K = [C_{R2} \times N / MAX] + 1$, where C_{R2} is the redundancy coefficient. For the traditional K-means algorithm, an additional step is required to ensure that the number of viewpoints in each cluster does not exceed the maximum limit. The point farthest from the center is reclassified into the nearest cluster if the number of viewpoints in the cluster exceeds MAX after one point is divided into this cluster. Furthermore, the results are output only when the number of viewpoints in each cluster does not exceed the maximum.

The initial cluster centers are selected randomly in K-means, and this initialization choice will directly affect the performance of the algorithm (Ismkhan 2018). This will result in the clustering falling into a local optimum without obtaining a globally optimal solution if the centers of the initial clusters are not chosen appropriately. Ideally, the number of viewpoints in each cluster after viewpoint clustering is the same in order, to fully utilize the computational capability of each thread and thus improve the overall efficiency of the algorithm. The overall grid scope is uniformly divided into K regions by area to locate the centers as far away from each other as possible. Then, the geometric centers of these regions are selected as the initial clustering centers. For

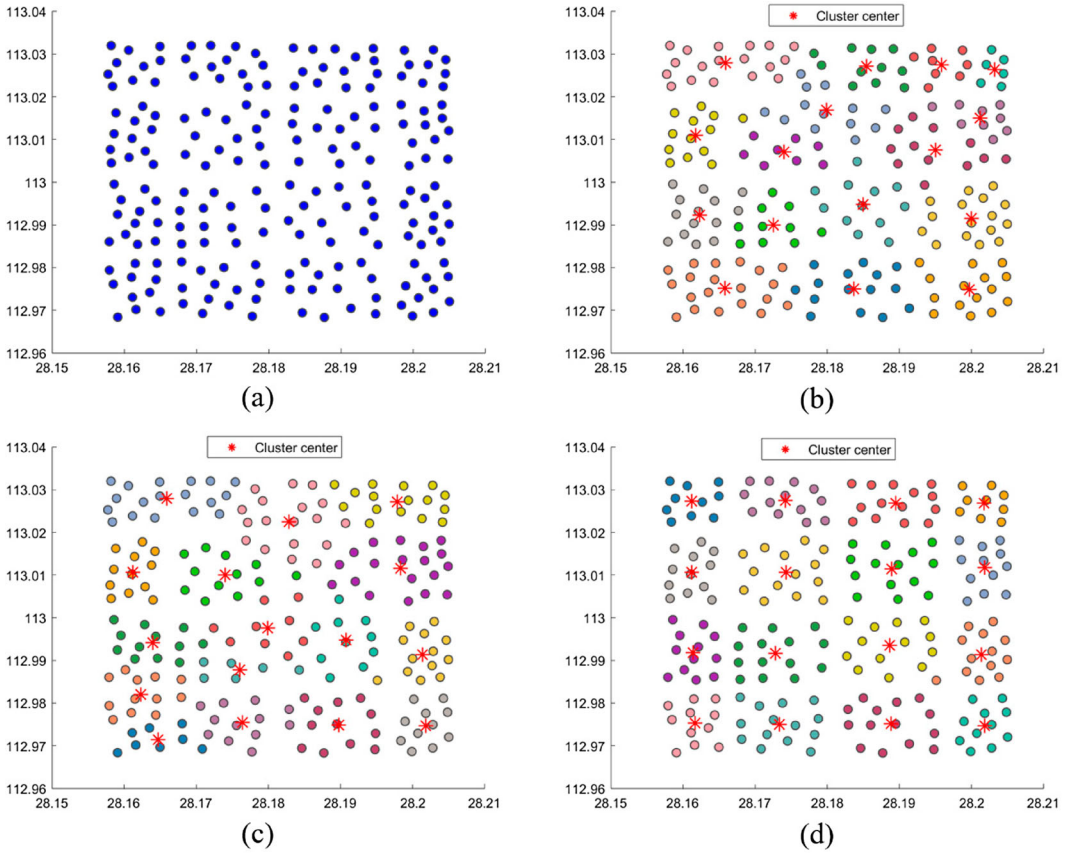


Figure 3. Clustering results under different initial clustering center conditions. (a) The original unclustered viewpoints; (b) the result of the traditional K-means algorithm; (c) the result of the K-means++ algorithm; (d) the result with the geometrical center of the divided area as the initial clustering center.

example, the total number of viewpoints is 300, and K is taken as 16. The clustering analysis is performed using the traditional K-means algorithm, the K-means++ algorithm, and our algorithm (Figure 3), where '*' indicates the center position of each cluster in the results. It is clear from the comparison that using the geometric centers of these regions as the initial clustering centers leads to results that are closer to the ideal, with the viewpoints divided more evenly between the clusters.

Upon completion of clustering, the grid occupied by each cluster can be determined based on the position of frustums within the cluster to the unit grid. The neighborhoods of the unit grids inside each cluster are already included. Only the neighborhood grids of the outermost unit grids need to be additionally divided into clusters. Grids are stored by clusters, and then viewshed analysis processes corresponding to each cluster are created, where the grids and the information contained within them are transmitted to the fragment shader through Uniform.

3.3. Adaptive depth bias

Determining whether a point is occluded is simply a matter of determining whether there is an object closer to the light source on the line connecting the point to the light. Here, the distance of the point from the light source is defined as the depth in the shadow map algorithm. The shadow map algorithm can be divided into 2 steps.

Step 1: The camera is positioned at the light source to render the whole scene and generate the depth Z_A. The depths of all points are stored in the depth texture, and the projection transformation matrix M is also recorded.

Step 2: The camera is repositioned at the observer position to render the entire scene. Point P is transformed from the world coordinate system to the light source coordinate system with M. Then, the two-dimensional coordinates (X_B, Y_B) and the depth Z_B are separated from the coordinates (X_B, Y_B, Z_B) of P. The original depth Z_A is sampled from the depth texture according to (X_B, Y_B). Z_B greater than Z_A means that there is an object obscuring point P from the light source and the point is in the shadow, whereas $Z_B = Z_A$ means that there is no object between point P and the light source so the point is in light (Figure 4).

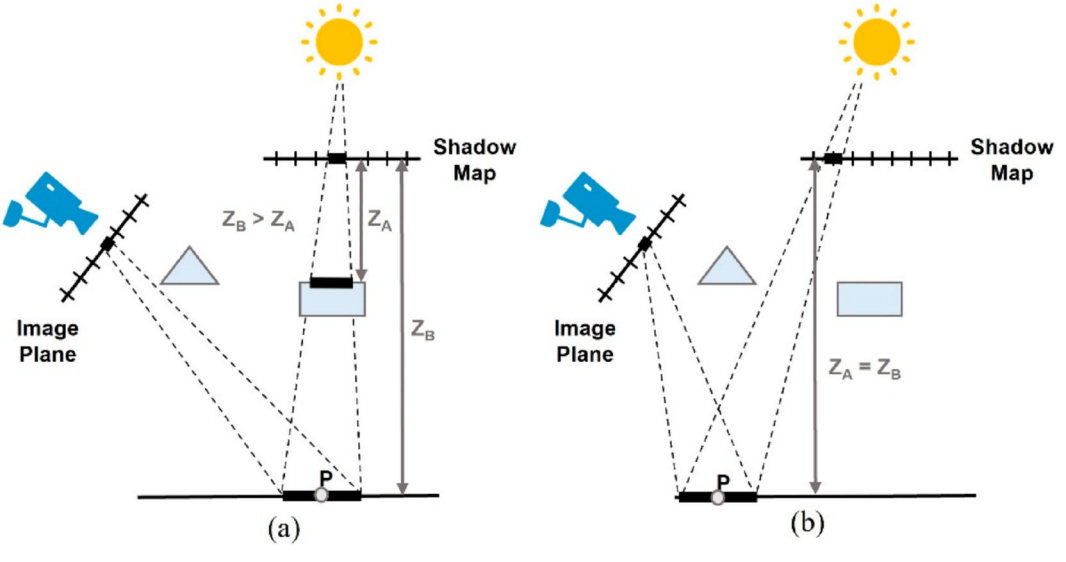


Figure 4. Schematic diagram of the shadow map. (a) An occluder is present between point *P* and the light source ($Z_B > Z_A$); (b) there are no obstacles between point *P* and the light source ($Z_B = Z_A$).

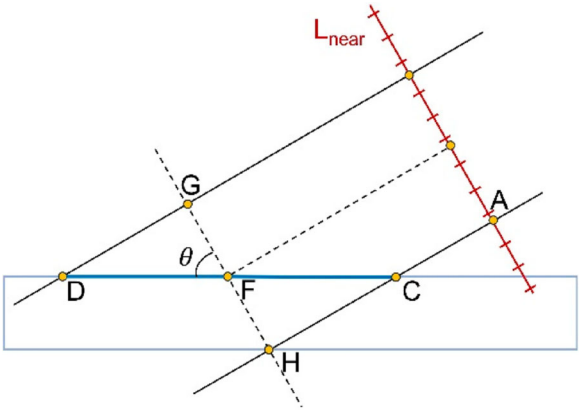


Figure 5. Schematic diagram of the Z-fighting phenomenon. The DF region is in the shadow, while the opposite is observed for the CF region. Z-fighting appears when multiple such regions occur in succession.

A single pixel in the shadow map may correspond to multiple pixels in the object plane, due to the numerical precision of depths and the limited resolution of shadow maps. Among pixels, some have larger depths, and others have smaller depths. Therefore, the result will exhibit shadow acne, a phenomenon known as Z-fighting, which usually occurs when two depths are very close. Assuming that the line segment AB represents the corresponding size of a unit pixel of the shadow map on the near clip plane, pixels within the scope of CD on the object are projected onto the AB region on L_{near} (Figure 5), where θ is the angle between the incident ray and the normal vector of the object plane. A perpendicular of AB is created through the midpoint of CD. Then, the value of AB as a single pixel in the depth map is |FE|. For any point X on the CD, the CD region at this time will appear as a half-white, half-black shadow acne because the distance d from X to L_{near} varies with the position of X:

$$\mathrm{d}\begin{cases} < |\mathrm{FE}|\,, & \mathrm{X} \in \mathrm{CF} \\ > |\mathrm{FE}|\,, & \mathrm{X} \in \mathrm{DF} \end{cases} \rightarrow \quad \text{visibility} = \begin{cases} \text{visible} & , \ \mathrm{X} \in \mathrm{CF} \\ \text{invisible} & , \ \mathrm{X} \in \mathrm{DF} \end{cases} \tag{1}$$

To eliminate Z-fighting, the points on the FD must be moved at a certain distance toward the light source. The distance of the movement is nominated as the depth bias. To avoid excessive redundant calculations, the majority of 3D engines do not calculate the bias value for each point exactly. Instead, they set a minimum bias that will satisfy all points on the CD based on the dimensions of the frustums and shadow maps in the scene. The minimum bias in this example is the depth bias of the critical point D:

$$\mathrm{DepthBias(D)} \ = \ |\mathrm{DG}| \ = \ \frac{|\mathrm{AB}|}{2} \times \tan(\theta) \tag{2}$$

It is known that $|AB| = S_{frustum} / S_{shadow}$, where $S_{frustum}$ is the size of the near plane of the frustum and S_{shadow} is the unit size of the shadow map. These parameters are fixed in each scene, so that |AB| can be regarded as constant in the same scene. The value of depth bias at point D is related to θ. In the extreme case when θ is close to 90° which means that the incident ray is almost parallel to the plane, the bias will tend to infinity. However, excessive bias will result in the light leakage phenomenon, which refers to the area behind the occlusion covered by the shadow being wrongly identified as light. Developers commonly set an upper limit on the bias to avoid this. Meanwhile, θ is not the only parameter that affects the bias. Areas further away from the light source are more prone to

shadow acne because the accuracy of storing the depth is limited. Therefore, the distance d_{light} between the point and the light source also must be taken into account.

Assuming that the angle of the incident ray is φ, then $\theta = \frac{\pi}{2} - \varphi$. Let the set of θ be $AN = \left\{\frac{\pi}{180}, \frac{\pi}{72}, \frac{\pi}{36}, \frac{\pi}{18}, \ldots, \frac{4\pi}{9}\right\}$ and the set of maximum visible distances be SD = {50, 100, … , 500}. For $d_{light} \in SD \wedge \theta \in AN$, set the depth bias and test the effects according to the traditional manual setting method in turn, and record the minimum depth bias that can eliminate the shadow acne for each condition. Our algorithm uses binary polynomial regression to establish a binary nonlinear regression model between d_{light} and θ because their effects on the bias are nonlinear, and obtains the regression equations:

$$\text{DepthBias} = p(d_{light}, \theta) \tag{3}$$

3.4. Customized fragment shader

Web Graphics Library (WebGL) can render high-performance interactive 3D graphics in any compatible browser (Kang and Lee 2019). Cesium is a WebGL-based open-source 3D mapping framework for creating virtual scenes of the 3D geographic information platform. There is a series of coordinate spaces in computer graphics (Figure 6), where the coordinates of an object start with local coordinates, followed by world coordinates, view coordinates, clip coordinates, and finally ending in screen coordinates (Goldman 2002). The transformation from one coordinate space to another is realized through matrices, of which the three most important transformation matrices are the model matrix, view matrix, and projection matrix.

The rendering pipeline is the parallel processing units within the GPU that are independent of each other to process graphics signals. It consists of two main functions: transformation of objects from the 3D coordinates of the scene to the 2D coordinates of the screen space and shading each pixel point in the screen. The most important parts of the rendering pipeline are the vertex shader and the fragment shader (Figure 7), because developers can customize their content through OpenGL Shading Language (GLSL) to achieve their goals.

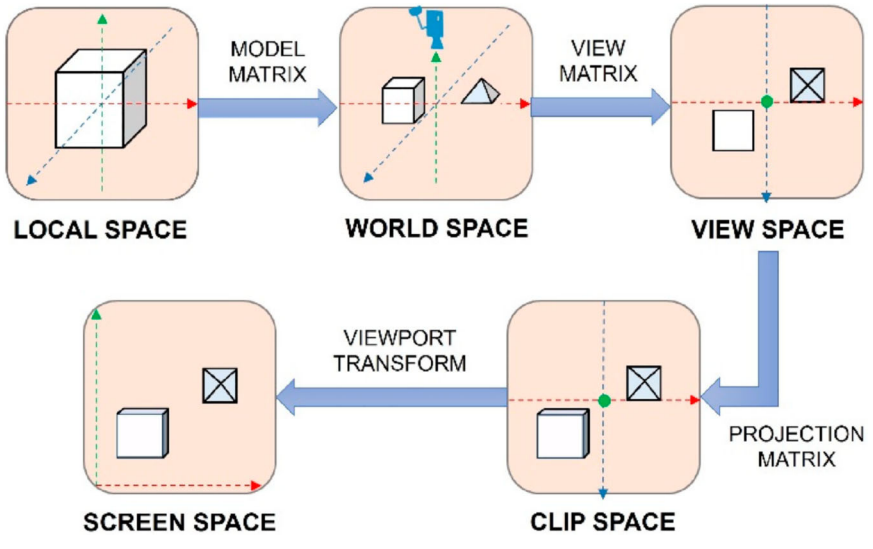


Figure 6. Graphical illustration of coordinate spaces and their sequence of transformations between each other. The coordinate systems are transformed by transformation matrices, and the corresponding inverse matrices are used when inverting the transformation.

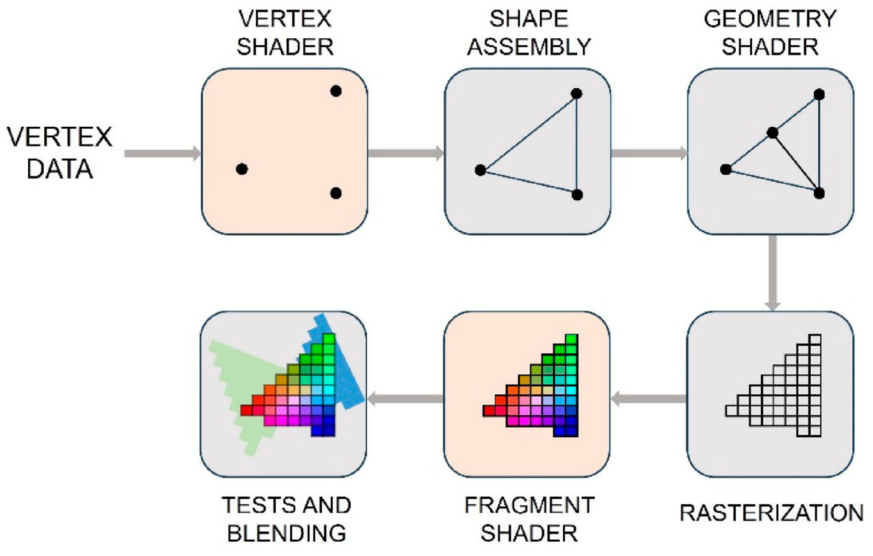


Figure 7. The core flow of the GPU rendering pipeline. Our algorithm is primarily implemented in the fragment shader within the pipeline.

Depth textures must be generated prior to the implementation of viewshed analysis in the fragment shader, because of the subsequent need for depth comparisons. The point light source used in this study generates omnidirectional shadow maps, making it necessary to create a corresponding cube map as a depth texture. A cube map is a collection of six separate square 2D textures that renders the entire scene to each face of a cube and samples it as surrounding depths. Any point on the cube map can be indexed by a direction vector (Figure 8).

Most GPUs are based on the single instruction multiple data (SIMD) or its extension single instruction multiple thread (SIMT) architectures. The chip consists of several small, lightweight cores that run their shader programs independently on each rendering pipeline, which is highly parallelized. The main function of the fragment shader is to calculate the final color of an individual pixel, and its processing object is the fragment. The core part of our algorithm is mainly implemented in the shader and can be summarized in the following steps (Algorithm 1).

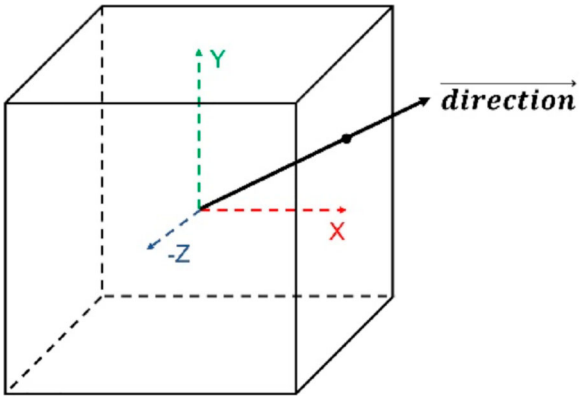


Figure 8. Schematic of cube map texture sampling. Assuming the geometric center of a unit cube as the origin, there exists a direction vector starting at the origin. The intersection of the vector and the surface of the cube is the corresponding texture sampling point.

Algorithm 1. A parallel algorithm for multi-viewpoint viewshed analysis based on target cluster segmentation.

Input	Texture coordinate V_{tex} of fragment P, screen coordinate (u, v), color texture, and depth texture.
Step1	Obtain the fragment's color V_{color} from the color texture according to V_{tex}, reconstruct the fragment's float-type depth f_Z from the depth texture, and calculate the z of the fragment in NDC space: $Z_{NDC} = 2f_Z - 1$ (4)
Step2	Calculate view coordinate V_{view} (X_V, Y_V, Z_V, W_V) of the fragment from the screen coordinate, and normalize it to the homogeneous coordinate: $V_{view} = M_{proj}^{-1} \times (2u - 1,\ 2v - 1,\ Z_{NDC},\ 1)^T$ (5) $V_{view}' = \left(\frac{X_v}{W_v},\ \frac{Y_v}{W_v},\ \frac{Z_v}{W_v},\ 1\right)$ (6) where M_{proj}^{-1} is the inverse matrix of the 4-dimensional projection transformation matrix.
Step3	Calculate the world coordinates of the fragment: $V_{world} = M_{view}^{-1} \times V_{view}'$ (7) where M_{view}^{-1} is the inverse matrix of the 4-dimensional view transformation matrix.
Step4	Let the set of grids corresponding to each cluster be $G = \{G_1, G_2, \ldots, G_n\}$. Calculate the grid number g corresponding to the fragment, and the grid cluster G_i to which it belongs according to V_{world}. If $g \in G_i$, execute the next step; if $g \notin G_i$, end the fragment shader for this fragment.
Step5	Determine the neighborhood grids (3-neighborhood, 5-neighborhood, or 8-neighborhood) of the fragment, based on the relative position of g to the overall grid boundary.
Step6	Iterate over the viewpoints in the neighborhood grids of the fragment. Determine whether the fragment is in the viewpoints' field of view, according to the distance and azimuth to the viewpoints. Then, obtain the m viewpoints associated with the fragment.
	For (j = 0; j < m; j++)
Step7	Transform the coordinate of the fragment from world space to clip space, corresponding to the elimination of all fragments outside the scope of the frustum F_j: $V_{clip} = M_{cam.proj} \times M_{cam.view} \times V_{world}$ (8) where $M_{cam.view}$ and $M_{cam.proj}$ are the 4-dimensional view transformation matrix and projection transformation matrix of the camera corresponding to viewpoint j.
Step8	Transform V_{clip} into V_{clip}' (X_c', Y_c', Z_c', 1) by homogeneous coordinate normalization. Then, determine whether V_{clip}' belongs to the NDC space, which means whether the fragment is within the frustum F_j: $P \begin{cases} \in F_j, & X_c' \in [-1,\ 1] \wedge Y_c' \in [-1,\ 1] \wedge Z_c' \in [-1,1] \\ \notin F_j, & \text{else} \end{cases}$ (9)
Step9	If $P \in F_j$, use Algorithm 2 to calculate the visibility index VI_j of viewpoint j to the fragment.
	End loop
Step10	Calculate the composite visibility index of the fragment: $VI = \sum_{j=1}^{m} VI_j$, and recalculate the color of the fragment based on the VI: $V_{Color}' = \begin{cases} \text{mix}(V_{Color},\ \text{LIME},\ 0.75), & VI \geq 1 \\ \text{mix}(V_{Color},\ \text{RED},\ 0.75), & VI = 0 \end{cases}$ (10) where LIME (0, 255, 0, 1) is the color of the visible area and RED (255, 0, 0, 1) is the color of the invisible area.
Output	Fragment's color V_{Color}'.

Algorithm 2. Algorithm for the visibility index of a single fragment based on shadow map.

Input	View coordinate V_{view}' of the fragment, view coordinate L_{view} of the viewpoint, and shadow map texture $cubeMap_j$.
Step1	V_{view}' (X_v', Y_v', Z_v', 1) is equivalent to V_{view}'' (X_v', Y_v', Z_v'). Calculate the direction vector in the view space pointing from the viewpoint to the fragment: $\overrightarrow{dir_{view}} = V_{view}'' - L_{view}$ (11)
Step2	Normalize $\overrightarrow{dir_{view}}$ to $\overrightarrow{dir_{view}}'$, and transform it from view space to world space: $\overrightarrow{dir_{world}} = M_{viewR}^{-1} \times \overrightarrow{dir_{view}}'$ (12) where M_{viewR}^{-1} is the inverse matrix of the 3-dimensional view rotation matrix.
Step3	Obtain the distance between the fragment and the viewpoint: $d = \lvert\overrightarrow{dir_{view}}\rvert$, and then calculate the depth of the scene: $Z_B = \frac{d}{r}$ (13) where r is the maximum visible distance of the viewpoint.
Step4	Calculate the depth bias based on the adaptive depth regression equation, and use it to offset the scene's depth: $Z_B' = Z_B - p(d,\ \theta)$ (14)
Step5	Obtain V_{depth} of the fragment from the depth texture according to the intersection of $\overrightarrow{dir_{view}}'$ and $cubeMap_j$, and convert it to a normalized float type: $Z_A = V_{depth} \cdot \left(1,\ \frac{1}{255},\ \frac{1}{255^2},\ \frac{1}{255^3}\right)^T$ (15)

Step6 Compare Z_A and Z_B values to calculate the visibility index of viewpoint j to the fragment:

$$VI_j = \begin{cases} 0, & Z_B > Z_A \\ 1, & \text{else} \end{cases} \quad (16)$$

Output Visibility index VI_j of viewpoint j to the fragment.

4. Experiments and discussion

4.1. Experiments and environment

This experiment is based on WebGL and JavaScript, and the core part of the algorithm is implemented in the fragment shader of the GPU rendering pipeline by GLSL. The final results are presented on Cesium, a 3D geographic visualization platform. To avoid the performance bottleneck of GPU affecting the accuracy of the results, and to compare the efficiency of the algorithms on GPUs with different performances, two PC platforms are selected for the comparison experiments (Table 1). Their system environment is Windows 10 64-bit. While there is a significant difference in the GPU performance between the platforms, the rest of the hardware specifications are basically the same. The DEMs used for the experiments were high-precision terrain-corrected ALOS remote sensing images with 12.5 m spatial resolution, and the image map data were Bing Maps at LOD 1-19. DEMs and image maps were offline data published by a local server to avoid the influence of the network environment on the experimental results.

3D Tiles is a tile data format for 3D models defined by Cesium for streaming and rendering against massive, heterogeneous 3D geospatial datasets, and is one of the OGC standards. This study focuses on large-scale viewshed analysis, and the fineness of the models has less influence on the results. Considering the model loading efficiency, the building models in this experiment adopt the simple model of 3D Tiles. The models must be generated prior to the experiment. First, the shapefile layer of Changsha city buildings was loaded in QGIS to merge the erroneously separated neighboring buildings. Then the layer was imported into ArcGIS Pro, and the flat building was stretched into a 3D shape based on the field of height. Finally, the stretched building layer and the DEMs were imported into the Cesiumlab tool as well. The elevations of the bottoms of the buildings were set according to the DEMs so that they could fit perfectly with the ground. Then, it was exported as a 3D Tiles building model.

Wireless communication viewshed analysis and the site selection problem are an important class of problems in viewshed analysis. They are often faced with large-scale, multi-viewpoint urban scenes in reality, matching well with the research focus of this study. The central city of Changsha, China, which is approximately 43.6 square kilometers and contains approximately 17,000 buildings, was selected as the experimental area for this experiment to verify the feasibility and efficiency of the algorithm. Within this area, 15, 45, 75, 150, 300, and 600 viewpoints were simulated for viewshed analysis according to the pattern of the wireless communication base station, which has a maximum viewing distance of 350 m, a horizontal field of view of 75°, a vertical field of view of 60°, and a downward tilt of 12°. Every three viewpoints formed a group of base stations with an inner group distance of 0.1 m to closely match the realistic scene. Viewpoints within the group could be approximated as the same position on a large scale.

Table 1. Configuration parameters of the experimental platforms. The only significant difference between the platforms is found for the GPU performance, whereas the rest of the parameters are essentially the same.

Platform	No. 1	No. 2
CPU	AMD 6850HS	AMD 5700X
CPU frequency	3.2-4.7 GHz	3.4-4.6 GHz
RAM	32 GB	32 GB
GPU	Radeon 680M	GTX 1080Ti
GPU memory	4 GB	11 GB

4.2. Experimental results

The shadow map viewshed algorithm and our algorithm were used to perform the viewshed analysis of the study area with the simulated viewpoints described above. Then, the computation times of the two algorithms were recorded. A detailed efficiency comparison of the shadow map viewshed algorithm with the traditional LOS-based algorithm has been performed in previous studies, and this part of the validation is not conducted in this study. The viewshed analysis was performed after the basic data such as image maps, DEMs, and 3D building models had been loaded. To ensure the accuracy of the algorithm's time consumption, average values of 5 experiments for each situation were taken as the final results (Table 2).

In scenes with 15, 45, and 75 viewpoints, the shadow map viewshed algorithm was always slightly more efficient than our algorithm, regardless of the platform. Even if the difference in the computation time between the algorithms was small, when the number of viewpoints was 150, 300, and 600, all of the results for the two platforms showed that our algorithm is significantly more efficient than the shadow map viewshed algorithm. In addition, the ratios for Platform 1 were higher than those for Platform 2. The 3D utilization of the GPU in the task manager was used as the occupancy, which can reflect well the occupancy of the traditional graphics performance of the GPU. During the algorithm operation, the occupancy of GPUs in the two platforms remained above 98%. The computation time increased with the number of viewpoints in both platforms, but there was a difference in the scaling of the time with the number of viewpoints between the two platforms. The polynomial fit curves of the algorithm's computation time and the number of viewpoints (Figure 9) showed that for the few viewpoints, the fit curves of the platforms almost overlap. However, as the number of viewpoints continuously increased, the computation time rose sharply for Platform 1.

In various traditional viewshed analysis algorithms, the scope of the field of view is the factor that has the greatest impact on the computation time other than the number of viewpoints. While the field of view is affected by parameters such as the height of the viewpoint, field of view angle, and downward tilt angle, the maximum visible distance plays a key role. Based on the above experiment, the maximum visible distance of the viewpoints was changed to 150 m, and the other parameters were kept the same. Viewshed analysis was performed again in Platform 1 using our algorithm, to obtain the average algorithmic time in the case of different numbers of viewpoints (Table 3). The maximum visible distance of some realistic application scenes typically reaches several kilometers, such as landscape protection for renewable energy projects (Möller 2010; Rodrigues, Montañés, and Fueyo 2010; Romanos Ioannidis and Koutsoyiannis 2020). Several viewpoints were simulated for experiments in nonurban areas where buildings are sparse (Table 3). The maximum visible distance of the viewpoints was set to 2500 m, with a mutual distance of 1000 m. The comparison indicates that the computation time varies in scenes with different maximum visible distances, but the difference is not significant.

The following is an example of the results of the viewshed analysis with 300 viewpoints. It is presented from different perspectives (Figures 10 and 11), where green is the visible area and red is the

Table 2. Comparison of time consumption (ms) between our algorithm and the shadow map viewshed algorithm on different platforms. With numerous viewpoints, our algorithm is significantly more efficient than the shadow map viewshed algorithm.

	Platform No. 1			Platform No. 2		
Number of viewpoints	Shadow Map	Our algorithm	Improvement ratio	Shadow Map	Our algorithm	Improvement ratio
15	7.220	7.274	−0.747%	5.090	5.225	−2.567%
45	12.424	12.446	−0.174%	8.780	9.175	−4.306%
75	28.540	28.576	−0.127%	19.152	19.160	−0.040%
150	177.260	165.021	7.417%	39.406	37.498	5.087%
300	319.542	290.899	9.847%	78.345	73.416	6.714%
600	547.086	496.437	10.202%	162.185	150.324	7.891%

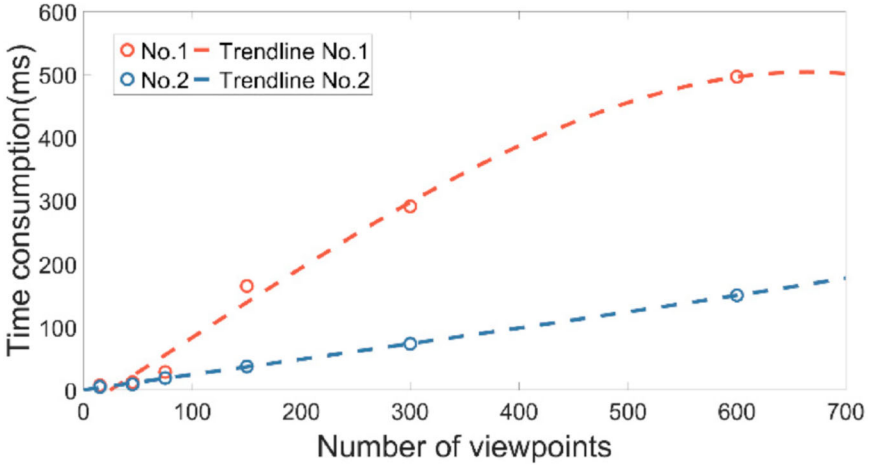


Figure 9. Polynomial fit curves of the time consumption of our algorithm and the number of viewpoints for the two platforms. As the viewpoint increases, the curves become increasingly different from each other.

invisible area. The overall graphs show that there is no difference between the results produced by the two algorithms. However, by comparing the resultant graphs from different perspectives of the local area, it is evident that some minor acne is present in the results obtained by the shadow map viewshed algorithm. For example, a small amount of red stripes or dotted shadows in the large green area indicates that small invisible areas mistakenly appeared in the location where the visible area should be found. By contrast, in the analyzed result graphs of our algorithm, such errors hardly ever occur.

4.3. Discussion

The experimental results indicate that our algorithm improves in efficiency compared to the shadow map viewshed algorithm in a large-scale, multi-viewpoint complex 3D scene. The improvement ratio rises with the increasing number of viewpoints. Previous studies have demonstrated that the shadow map viewshed algorithm is significantly more efficient than the traditional LOS-based algorithms (Chao et al. 2011; Chen and Chen 2021), also indicating that our algorithm is superior to the traditional viewshed analysis algorithms concerning efficiency. Target cluster segmentation in the algorithm plays a key role in efficiency improvement. The uniform grid division and K-means spatial clustering methods in our algorithm can avoid multiple repetitive operations on a single fragment under numerous invalid light sources. Meanwhile, target cluster-based segmentation enables clusters to be analyzed in parallel, thus utilizing the parallel computing power of GPUs more efficiently.

Table 3. Comparison of the time consumption (ms) of our algorithm for different maximum visible distance conditions in Platform 1. There is no significant difference between 150 and 350 m. The time consumption at 2500 m is lower due to the sparse buildings.

	Urban areas		Nonurban areas
Number of viewpoints	Sight distance of 150 m	Sight distance of 350 m	Sight distance of 2500 m
15	6.675	7.274	5.129
45	11.598	12.446	14.225
75	26.247	28.576	27.073
150	160.368	165.021	51.727
300	279.231	290.899	112.276
600	494.714	496.437	235.045

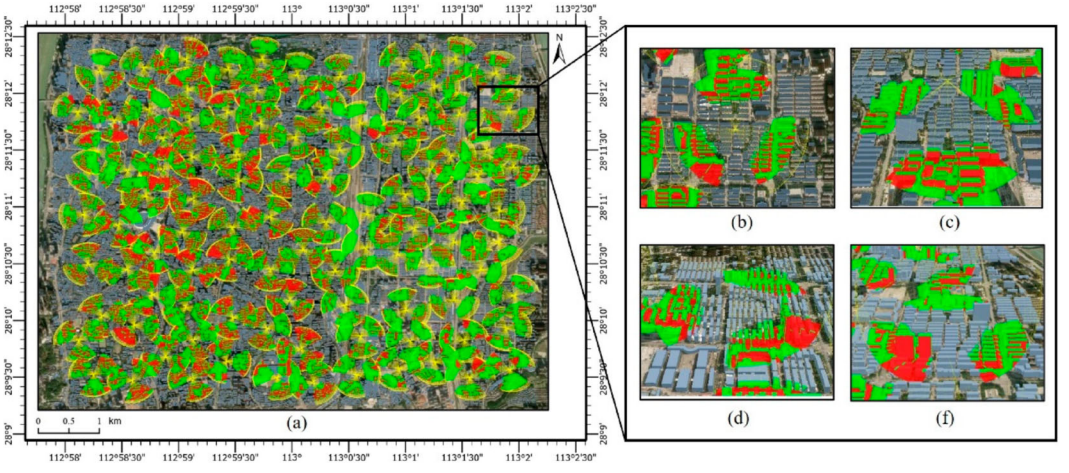


Figure 10. Results of viewshed analysis by the shadow map viewshed algorithm for a scene with 300 viewpoints. (a) The results for the study area as a whole, and (b), (c), (d), and (e) the results from different perspectives of a certain local area from the top vertical, due north, due west, and due south, respectively.

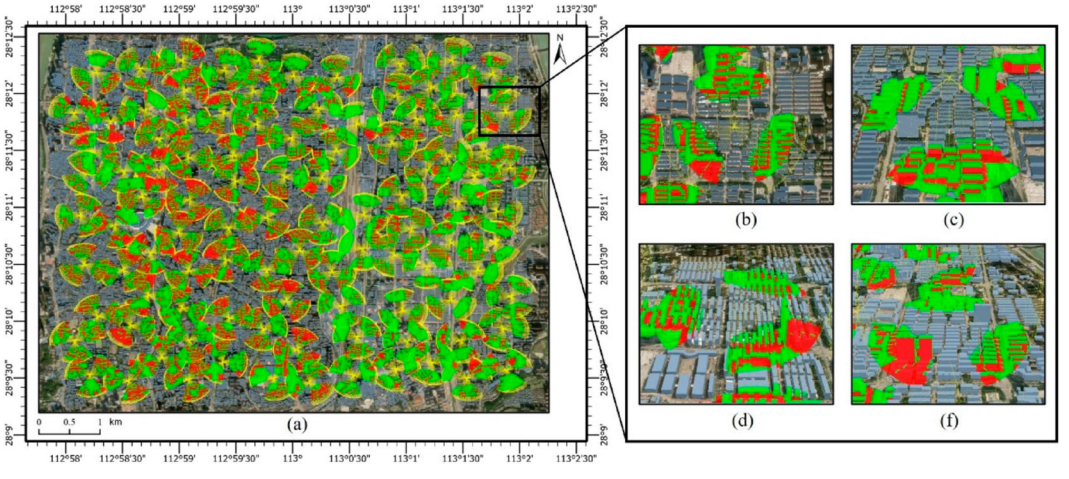


Figure 11. Results of viewshed analysis by our algorithm for a scene with 300 viewpoints. (a) The results for the study area as a whole, and (b), (c), (d), and (e) the results from different perspectives of a certain local area from the top vertical, due north, due west, and due south, respectively.

Comparing the graphs of the analysis results of the two algorithms proves the accuracy of our algorithm. However, as observed from the local detailed result graphs, our algorithm avoids the majority of minor errors that occur in the results of the shadow map viewshed algorithm, this is attributed to the adaptive depth bias method in the algorithm. Overall, our algorithm achieves some efficiency and accuracy improvements compared to the shadow map viewshed algorithm in large-scale, multi-viewpoint complex 3D scenes. Furthermore, the viewshed analysis algorithm based on target cluster segmentation is more efficient than the algorithm that is not based on such segmentation.

In traditional viewshed analysis algorithms, DEMs and geometric features increase with the expansion of the field of view, ultimately leading to a surge in computation and time. The time consumption of our algorithm with the condition that the maximum visible distance was 150 m was not significantly reduced compared to the scene with the maximum visible distance of 350 m, even though the field of view was reduced by approximately 80%. Similarly, time

consumption did not increase dramatically at 2500 m. This is consistent with previous findings on the effect of the field of view in the shadow map viewshed algorithm (Chao et al. 2011). The time consumption was markedly decreased in nonurban areas with sparse buildings, even in the 2,500 m conditions where the field of view is expanded several times. This also suggests that the field of view has little effect on the algorithm's time consumption. The number of buildings is an important factor that affects the algorithm's computation time, which is also consistent with the results of previous research (Chen and Chen 2021). In our algorithm, viewshed analysis does not require DEM interpolation and relies on the GPU rendering pipeline. Moreover, changes in the field of view do not affect the generation of cube shadow maps, and comparisons of depths have a small effect on the performance. Therefore, the field of view does not have much effect on the algorithm's time consumption. Since the maximum visible distance is the decisive factor affecting the scope of the field of view, it can be inferred that parameters such as viewpoint height, the field of view angles, and downward tilt angle have a similarly small effect on the algorithm's time consumption.

5. Conclusions

With the increase in the number of viewpoints, the computational resources needed for viewshed analysis in 3D scenes rise dramatically. This paper proposes a parallel algorithm for multi-viewpoint viewshed analysis based on target cluster segmentation for large-scale, multi-viewpoint complex 3D scenes, building on the shadow map algorithm. This algorithm achieves target cluster segmentation through uniform grid division and K-means spatial clustering. It is compatible with the terrain and features while fully utilizing the parallel computational capability of GPU and avoiding DEM interpolation operations, and provides fast and accurate viewshed analysis of complex 3D scenes. Experiments demonstrate that our algorithm is significantly more efficient than the shadow map viewshed algorithm not based on target clusters for scenes with a significant number of viewpoints. This is particularly true when the GPU performance is bottlenecked. Moreover, the field of view has little effect on the computation time. According to the comparisons made in previous studies, the efficiency of our algorithm is also superior to the traditional LOS-based algorithms. Benefiting from the adaptive depth bias method in the algorithm, our algorithm improves the efficiency and accuracy of the viewshed analysis. This algorithm fits well with many realistic project scenes and has broad potential application prospects.

However, our algorithm still has some shortcomings. The first is that the uniform grid division and spatial clustering process increases some time cost, making the algorithm's efficiency not outstanding in scenes with a few viewpoints. The second is that the algorithm is primarily implemented in the GPU rendering pipeline, so the interaction of the analyzed results with other systems is not as simple as it would be for the traditional algorithms executed in the CPU. Further algorithm optimization to address these issues will be considered in future work.

Acknowledgements

The authors would like to thank anonymous peer reviewers for helpful and constructive reviews of the manuscript's content.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by National Natural Science Foundation of China [grant number 42172330].

Data availability statement

The data that support the findings of this study are available from the corresponding author, upon reasonable request.

ORCID

Tianren Xiao http://orcid.org/0009-0004-5937-0759
Jiqiu Deng http://orcid.org/0000-0003-0729-1018
Chengfeng Wen http://orcid.org/0009-0005-7387-8687
Qiqi Gu http://orcid.org/0009-0005-0180-1322

References

Alzubaidi, Laith, Jinglan Zhang, Amjad J. Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, J. Santamaría, Mohammed A. Fadhel, Muthana Al-Amidie, and Laith Farhan. 2021. "Review of Deep Learning: Concepts, CNN Architectures, Challenges, Applications, Future Directions." *Journal of Big Data* 8 (1): 53. https://doi.org/10.1186/s40537-021-00444-8.

Annoni, Alessandro, Stefano Nativi, Arzu Çöltekin, Cheryl Desha, Eugene Eremchenko, Caroline M. Gevaert, Gregory Giuliani, et al. 2023. "Digital Earth: Yesterday, Today, and Tomorrow." *International Journal of Digital Earth* 16 (1): 1022–1072. https://doi.org/10.1080/17538947.2023.2187467.

Baek, Jieun, and Yosoon Choi. 2018. "A New GIS-Based Algorithm to Support Initial Transmitter Layout Design in Open-Pit Mines." *Energies* 11 (11): 3063. https://doi.org/10.3390/en11113063.

Bao, Shitai, Ningchuan Xiao, Zehui Lai, Heyuan Zhang, and Changjoo Kim. 2015. "Optimizing Watchtower Locations for Forest Fire Monitoring Using Location Models." *Fire Safety Journal* 71 (January): 100–109. https://doi.org/10.1016/j.firesaf.2014.11.016.

Beer, Martin, Radim Rybár, and Ľubomíra Gabániová. 2023. "Visual Impact of Renewable Energy Infrastructure: Implications for Deployment and Public Perception." *Processes* 11 (8): 2252. https://doi.org/10.3390/pr11082252.

Cauchi-Saunders, Aran J., and Ian J. Lewis. 2015. "GPU Enabled XDraw Viewshed Analysis." *Journal of Parallel and Distributed Computing* 84 (October): 87–93. https://doi.org/10.1016/j.jpdc.2015.07.001.

Chang, C. T., B. Gorissen, and S. Melchior. 2011. "Fast Oriented Bounding Box Optimization on the Rotation Group SO(3, R)." *ACM TRANSACTIONS ON GRAPHICS* 30 (5). https://doi.org/10.1145/2019627.2019641.

Chao, Fang, Yang Chongjun, Chen Zhuo, Yao Xiaojing, and Guo Hantao. 2011. "Parallel Algorithm for Viewshed Analysis on a Modern GPU." *International Journal of Digital Earth* 4 (6): 471–486. https://doi.org/10.1080/17538947.2011.555565.

Chen, Yuxuan, and Jing Chen. 2021. "A Parallel Multipoint Viewshed Analysis Method for Urban 3D Building Scenes." *Transactions in GIS* 25 (4): 2010–2028. https://doi.org/10.1111/tgis.12759.

Cimburova, Zofie, and Stefan Blumentrath. 2022. "Viewshed-Based Modelling of Visual Exposure to Urban Greenery – An Efficient GIS Tool for Practical Planning Applications." *Landscape and Urban Planning* 222 (June): 104395. https://doi.org/10.1016/j.landurbplan.2022.104395.

De Floriani, Leila, Paola Marzano, and Enrico Puppo. 1994. "Line-of-Sight Communication on Terrain Models." *International Journal of Geographical Information Systems* 8 (4): 329–342. https://doi.org/10.1080/02693799408902004.

Feng, Wang, Wang Gang, Pan Deji, Liu Yuan, Yang Liuzhong, and Wang Hongbo. 2015. "A Parallel Algorithm for Viewshed Analysis in Three-Dimensional Digital Earth." *Computers & Geosciences* 75 (February): 57–65. https://doi.org/10.1016/j.cageo.2014.10.012.

Goldman, Ron. 2002. "On the Algebraic and Geometric Foundations of Computer Graphics." *ACM Transactions on Graphics* 21 (1): 52–86. https://doi.org/10.1145/504789.504792.

Han, Zhigang, Songnian Li, Caihui Cui, Hongquan Song, Yunfeng Kong, and Fen Qin. 2019. "Camera Planning for Area Surveillance: A New Method for Coverage Inference and Optimization Using Location-Based Service Data." *Computers, Environment and Urban Systems* 78 (November): 101396. https://doi.org/10.1016/j.compenvurbsys.2019.101396.

Henrico, I., S. Henrico, and S. Coetzee. 2020. "A Comparison Between Two Dem Products To Calculate A Visibility Analysis For Military Operations Using Fossgis." *Geografia Fisica E Dinamica Quaternaria* 43 (1): 157–165. https://doi.org/10.4461/GFDQ.2020.43.6.

Ikotun, Abiodun M., Absalom E. Ezugwu, Laith Abualigah, Belal Abuhaija, and Jia Heming. 2023. "K-Means Clustering Algorithms: A Comprehensive Review, Variants Analysis, and Advances in the Era of Big Data." *Information Sciences* 622 (April): 178–210. https://doi.org/10.1016/j.ins.2022.11.139.

Ioannidis, Romanos, and Demetris Koutsoyiannis. 2020. "A Review of Land Use, Visibility and Public Perception of Renewable Energy in the Context of Landscape Impact." *Applied Energy* 276 (October): 115367. https://doi.org/10.1016/j.apenergy.2020.115367.

Ioannidis, R., N. Mamassis, A. Efstratiadis, and D. Koutsoyiannis. 2022. "Reversing Visibility Analysis: Towards an Accelerated a Priori Assessment of Landscape Impacts of Renewable Energy Projects." *Renewable and Sustainable Energy Reviews* 161 (June): 112389. https://doi.org/10.1016/j.rser.2022.112389.

Ismkhan, Hassan. 2018. "I-k-Means−+: An Iterative Clustering Algorithm Based on an Enhanced Version of the k-Means." *Pattern Recognition* 79 (July): 402–413. https://doi.org/10.1016/j.patcog.2018.02.015.

Jones, Eric E. 2006. "Using Viewshed Analysis to Explore Settlement Choice: A Case Study of the Onondaga Iroquois." *American Antiquity* 71 (3): 523–538. https://doi.org/10.2307/40035363.

Kang, Seokchan, and Jiyeong Lee. 2019. "Improving Rendering Speed of 3D Geospatial Data Based on HTML5/WebGL Using Improved Arithmetic Operation Speed." *International Journal of Urban Sciences* 23 (3): 303–317. https://doi.org/10.1080/12265934.2018.1476175.

Liu, Liu, LiQiang Zhang, JingTao Ma, Liang Zhang, XingMing Zhang, ZhiQiang Xiao, and Ling Yang. 2010. "An Improved Line-of-Sight Method for Visibility Analysis in 3D Complex Landscapes." *Science China Information Sciences* 53 (11): 2185–2194. https://doi.org/10.1007/s11432-010-4090-x.

Moll, Philipp, Selina Isak, Hermann Hellwagner, and Jeff Burke. 2021. "A Quadtree-Based Synchronization Protocol for Inter-Server Game State Synchronization." Computer Networks 185 (February): 107723. https://doi.org/10.1016/j.comnet.2020.107723.

Möller, Bernd. 2010. "Spatial Analyses of Emerging and Fading Wind Energy Landscapes in Denmark." *Land Use Policy* 27 (2): 233–241. https://doi.org/10.1016/j.landusepol.2009.06.001.

Murphy, Kathryn M., Bruce Gittings, and Jim Crow. 2018. "Visibility Analysis of the Roman Communication Network in Southern Scotland." *Journal of Archaeological Science: Reports* 17 (February): 111–124. https://doi.org/10.1016/j.jasrep.2017.10.047.

Owens, John D., David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E. Lefohn, and Timothy J. Purcell. 2007. "A Survey of General-Purpose Computation on Graphics Hardware." *Computer Graphics Forum* 26 (1): 80–113. https://doi.org/10.1111/j.1467-8659.2007.01012.x.

Park, Sang-Min, and Young-Gab Kim. 2022. "A Metaverse: Taxonomy, Components, Applications, and Open Challenges." *IEEE Access* 10: 4209–4251. https://doi.org/10.1109/ACCESS.2021.3140175.

Rodrigues, Marcos, Carlos Montañés, and Norberto Fueyo. 2010. "A Method for the Assessment of the Visual Impact Caused by the Large-Scale Deployment of Renewable-Energy Facilities." *Environmental Impact Assessment Review* 30 (4): 240–246. https://doi.org/10.1016/j.eiar.2009.10.004.

Rua, Helena, Alexandre B. Gonçalves, and Ricardo Figueiredo. 2013. "Assessment of the Lines of Torres Vedras Defensive System with Visibility Analysis." *Journal of Archaeological Science* 40 (4): 2113–2123. https://doi.org/10.1016/j.jas.2012.12.012.

Sanchez-Fernandez, Andres J., Luis F. Romero, Gerardo Bandera, and Siham Tabik. 2022. "VPP: Visibility-Based Path Planning Heuristic for Monitoring Large Regions of Complex Terrain Using a UAV Onboard Camera." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 15: 944–955. https://doi.org/10.1109/JSTARS.2021.3134948.

Sander, Heather A., and Steven M. Manson. 2007. "Heights and Locations of Artificial Structures in Viewshed Calculation: How Close Is Close Enough?" *Landscape and Urban Planning* 82 (4): 257–270. https://doi.org/10.1016/j.landurbplan.2007.03.002.

Sen, Pradeep, Mike Cammarano, and Pat Hanrahan. 2003. "Shadow Silhouette Maps." *ACM Transactions on Graphics* 22 (3): 521–526. https://doi.org/10.1145/882262.882301.

Sobala, Michał, Urszula Myga-Piątek, and Bartłomiej Szypuła. 2020. "Assessment of Changes in a Viewshed in the Western Carpathians Landscape as a Result of Reforestation." *Land* 9 (11): 430. https://doi.org/10.3390/land9110430.

Stamminger, M., and G. Drettakis. 2002. "Perspective Shadow Maps." *ACM Transactions on Graphics* 21 (3): 557–562. https://doi.org/10.1145/566654.566616

Wang, Meizhen, Xuejun Liu, Yanan Zhang, and Ziran Wang. 2017. "Camera Coverage Estimation Based on Multistage Grid Subdivision." *ISPRS International Journal of Geo-Information* 6 (4): 110. https://doi.org/10.3390/ijgi6040110.

Wang, J. J., G. J. Robinson, and K. White. 2000. "Generating Viewsheds Without Using Sightlines." *Photogrammetric Engineering And Remote Sensing* 66 (1): 87–90.

Welscher, Franz, Rizwan Bulbul, Johannes Scholz, and Peter Lederer. 2023. "The Antenna Coverage Location Problem in the Context of Cattle Tracking in the Austrian Alps." *International Journal of Applied Earth Observation and Geoinformation* 122 (August): 103414. https://doi.org/10.1016/j.jag.2023.103414.

Wm, Prof, Randolph Franklin, Ltc Clark, K. Ray, and Shashank Mehta. 1998. "Geometric Algorithms for Siting of Air Defense Missile Batteries," July.

Wu, Tianyu, Shizhu He, Jingping Liu, Siqi Sun, Kang Liu, Qing-Long Han, and Yang Tang. 2023. "A Brief Overview of ChatGPT: The History, Status Quo and Potential Future Development." *IEEE/CAA Journal of Automatica Sinica* 10 (5): 1122–1136. https://doi.org/10.1109/JAS.2023.123618.

Wu, Huanping, Mao Pan, Lingqing Yao, and Bing Luo. 2007. "A Partition-Based Serial Algorithm for Generating Viewshed on Massive DEMs." *International Journal of Geographical Information Science* 21 (9): 955–964. https://doi.org/10.1080/13658810601034218.

Yanlan, W. 2001. "An Algorithm Computing Viewsheds Based on Reference Planes." *WTUSM Bulletin of Science and Technology*. 1 (January): 19–21+25. https://doi.org/10.14188/j.2095-6045.2001.01.007.

You, Jiawei, Yongjian Huai, Xiaoying Nie, and Yuanyuan Chen. 2022. "Real-Time 3D Visualization of Forest Fire Spread Based on Tree Morphology and Finite State Machine." *Computers & Graphics* 103 (April): 109–120. https://doi.org/10.1016/j.cag.2022.01.009.

Yu, Siyi, Bailang Yu, Wei Song, Bin Wu, Jianhua Zhou, Yan Huang, Jianping Wu, Feng Zhao, and Weiqing Mao. 2016. "View-Based Greenery: A Three-Dimensional Assessment of City Buildings' Green Visibility Using Floor Green View Index." *Landscape and Urban Planning* 152 (August): 13–26. https://doi.org/10.1016/j.landurbplan.2016.04.004.

Zhao, Yanli, Anand Padmanabhan, and Shaowen Wang. 2013. "A Parallel Computing Approach to Viewshed Analysis of Large Terrain Data Using Graphics Processing Units." *International Journal of Geographical Information Science* 27 (2): 363–384. https://doi.org/10.1080/13658816.2012.692372.